Sprint 1 · Cimientos: apuntador a struct y paso por apuntador

Objetivo del sprint	Tener una `Idol` funcional que se manipula por apuntador: crearla, mostrarla, hacerla ensayar y descansar, y saber si sigue activa.
Etapas que cubre	Etapas 0, 1 y 2
Puntos comprometidos	17 puntos en 9 historias
Incremento esperado	`manager.c` con las Etapas 1 y 2 completas, compilando sin advertencias.

Sprint Backlog

ID	Historia	Tareas sugeridas	Pts	Responsable
HU-00	Preparar entorno y tablero	Crear carpeta o repositorio · Compilar `main` vacío · Armar el tablero · Cargar tarjetas del Sprint 1 · Elegir Scrum Master	2	2
HU-01	Definir la estructura Idol	Declarar el `typedef struct` · Crear `jennie` en `main` · Compilar con `-Wall -Wextra`	2	2
HU-02	Leer campos con punto, desreferencia y flecha	Declarar `p` · Escribir los tres `printf` · Imprimir direcciones con `%p` · Comparar las salidas	3	3
HU-03	Mostrar una integrante	Escribir `mostrar` con `const` · Llamarla con `&jennie` · Probar que modificar un campo no compila	2	2
HU-04	Limitar un valor a un rango	Escribir `limitar` · Probar al menos cinco casos (dentro, extremos y fuera)	1	1
HU-05	Ensayar para ganar popularidad	Escribir `ensayar` · Llamarla 20 veces y revisar la energía · Confirmar que `main` ve los cambios	2	2
HU-06	Descansar para recuperar energía	Definir la cantidad de recuperación · Escribir `descansar` · Llamarla 20 veces y revisar el límite	2	2
HU-07	Saber si una integrante está activa	Escribir `esta_activa` · Probar con energía 0 y con energía 1	1	1
HU-08	Explicar paso por valor y paso por apuntador	Escribir una función de demostración por valor · Compararla con `ensayar` · Preparar la explicación oral	2	2
		Total	**17**	17

Sprint Review: se acepta el sprint si…

- Las tres formas de leer los fans imprimen el mismo valor y las dos direcciones `%p` coinciden.
- `mostrar(&jennie)` imprime todos los datos.
- Veinte llamadas a `descansar` nunca pasan de `energia_max` y veinte a `ensayar` nunca bajan la energía de 0.
- Cada integrante explica qué se copia al pasar un struct por valor y qué al pasar un apuntador.

Riesgos y consejos

- Confundir `.` con `->`: usar `.` con variables y `->` con apuntadores.
- Olvidar el `&` al llamar a las funciones (`ensayar(jennie)` en lugar de `ensayar(&jennie)`).

- Cuidado con dejar “sin const” a `mostrar`: la práctica pide `const Idol *`.

Etapa 0 · Preparación del equipo

Antes de programar, el equipo organiza su entorno de trabajo y su tablero. **Sprint 1.**

HU-00 · Preparar entorno y tablero	Prioridad Alta · 2 pts

Como equipo de desarrollo, **quiero** tener listos el archivo `manager.c`, el comando de compilación y el tablero Kanban, **para** empezar a trabajar de forma ordenada desde el primer día.

Criterios de aceptación

- Existe `manager.c` en una carpeta o repositorio compartido por todo el equipo.
- El comando `gcc -Wall -Wextra -o manager manager.c` compila un programa mínimo (`main` vacío) sin advertencias.
- El tablero tiene las columnas Por hacer, En progreso, En revisión y Hecho, con los límites WIP acordados.
- Las historias del Sprint 1 están cargadas como tarjetas en Por hacer.
- Se eligió al Scrum Master del Sprint 1 y quedó anotado en la portada.

Depende de: — · Sprint 1 · Entregable: manager.c vacío compilando + tablero Kanban

Etapa 1 · Una integrante

Concepto: apuntador a struct, operador `&` y acceso con `->`. **Producto:** una `Idol` y la función `mostrar` con `const`. **Sprint 1.**

HU-01 · Definir la estructura Idol	Prioridad Alta · 2 pts

Como manager del grupo, **quiero** registrar a una integrante con nombre, popularidad, energía actual, energía máxima y fans, **para** contar con los datos base del sistema.

Criterios de aceptación

- `Idol` está definida con `typedef struct` y tiene `nombre` (arreglo de 30 caracteres), `popularidad`, `energia`, `energia_max` y `fans`.
- En `main` se crea `jennie` con valores iniciales, por ejemplo `{"jennie", 50, 100, 100, 12000}`.
- La energía inicial no supera `energia_max`.
- El programa compila sin advertencias.

Depende de: HU-00 · Sprint 1 · Entregable: struct Idol + variable jennie

HU-02 · Leer campos con punto, desreferencia y flecha	Prioridad Alta · 3 pts

Como estudiante, **quiero** declarar un apuntador a una `Idol` e imprimir sus fans de tres maneras, **para** entender la relación entre struct, dirección y acceso a campos.

Criterios de aceptación

- Existe `Idol *p = &jennie;`.
- Los fans se imprimen con `jennie.fans`, con `(*p).fans` y con `p->fans`.
- Las tres impresiones muestran el mismo valor.
- Se imprimen con `%p` (convertidas a `(void *)`) la dirección guardada en `p` y la de `&jennie`, y ambas son iguales.

Depende de: HU-01 · Sprint 1 · Entregable: `Idol *p = &jennie;` + 3 formas de leer fans

HU-03 · Mostrar una integrante	Prioridad Alta · 2 pts

Como manager del grupo, **quiero** ver en pantalla todos los datos de una integrante, **para** revisar su estado en cualquier momento.

Criterios de aceptación

- Existe `void mostrar(const Idol *i)` que imprime nombre, popularidad, energía actual/máxima y fans.
- Se llama con `mostrar(&jennie)` y los datos mostrados son correctos.
- El parámetro lleva `const`: si dentro de la función se intenta modificar un campo, el compilador marca error.
- La función accede a los campos con `->`.

Depende de: HU-01 · Sprint 1 · Entregable: `void mostrar(const Idol *i)`

Etapa 2 · Paso por apuntador

Concepto: la función trabaja con el struct original, no con una copia. **Producto:** `limitar`, `ensayar`, `descansar` y `esta_activa`. **Sprint 1.**

HU-04 · Limitar un valor a un rango	Prioridad Alta · 1 pts

Como estudiante, **quiero** una función que mantenga un número entre un mínimo y un máximo, **para** evitar energías negativas o por encima del límite.

Criterios de aceptación

- Existe `int limitar(int valor, int minimo, int maximo)`.
- Devuelve `minimo` si `valor < minimo`, `maximo` si `valor > maximo` y `valor` en otro caso.
- Se prueba con valores dentro del rango, en los extremos y fuera de él (por ejemplo `limitar(-5, 0, 100)`, `limitar(150, 0, 100)` y `limitar(40, 0, 100)`).
- No usa variables globales.

Depende de: HU-01 · Sprint 1 · Entregable: `int limitar(int valor, int minimo, int maximo)`

HU-05 · Ensayar para ganar popularidad	Prioridad Alta · 2 pts

Como manager del grupo, **quiero** que una integrante ensaye, **para** que gane popularidad a cambio de energía.

Criterios de aceptación

- Existe `void ensayar(Idol *i)`: suma popularidad (referencia: +5) y resta energía (referencia: −20) usando `limitar`.
- Después de `ensayar(&jennie)`, los cambios se ven en la variable original de `main`, sin reasignar nada.
- Llamar 20 veces a `ensayar` nunca deja la energía por debajo de 0.
- El parámetro es un apuntador sin `const`, porque la función modifica el struct.

Depende de: HU-04 · Sprint 1 · Entregable: `void ensayar(Idol *i)`

HU-06 · Descansar para recuperar energía

Prioridad Alta · 2 pts

Como manager del grupo, **quiero** que una integrante descanse, **para** que recupere energía sin pasar de su máximo.

Criterios de aceptación

- Existe `void descansar(Idol *i)` que recupera una cantidad fija de energía (el equipo la define, por ejemplo 30).
- Usa `limitar` para que la energía siempre quede entre 0 y `energia_max`.
- Llamar 20 veces a `descansar` nunca supera `energia_max`.
- Si se alternan `ensayar` y `descansar`, `mostrar` refleja los valores correctos.

Depende de: HU-04 · Sprint 1 · Entregable: `void descansar(Idol *i)`

HU-07 · Saber si una integrante está activa

Prioridad Alta · 1 pts

Como manager del grupo, **quiero** consultar si una integrante todavía tiene energía, **para** decidir si puede seguir trabajando.

Criterios de aceptación

- `int esta_activa(const Idol *i)` devuelve 1 si `energia > 0` y 0 en otro caso.
- Devuelve 0 cuando la energía es exactamente 0 y 1 cuando es 1.
- El parámetro lleva `const` porque la función solo lee.

Depende de: HU-01 · Sprint 1 · Entregable: `int esta_activa(const Idol *i)`

HU-08 · Explicar paso por valor y paso por apuntador

Prioridad Media · 2 pts

Como estudiante, **quiero** explicar qué se copia al pasar un struct por valor y qué al pasar un apuntador, **para** justificar por qué las funciones de esta etapa reciben `Idol *`.

Criterios de aceptación

- El equipo escribe una demostración breve: una función que recibe `Idol` por valor y no cambia el original, junto a `ensayar`, que sí lo cambia.
- La explicación indica que por valor se copia todo el struct (incluido el arreglo `nombre`) y por apuntador solo se copia una dirección.
- Cada integrante del equipo puede dar la explicación en voz alta durante la Sprint Review.

Depende de: HU-05 · Sprint 1 · Entregable: Demostración + explicación oral